

JAVA

CORSO in java

DOCENTE: PIERPAOLO DELLA VECCHIA

JAVA - CARATTERISTICHE

- Il linguaggio JAVA viene rilasciato il 23 maggio del 1995 da Sun Microsystem assorbita poi da **Oracle** nel 2010
- **SINTASSI**: simile a quella dei linguaggi di programmazione storici quali c,c++
- **ROBUSTEZZA**: previene gli errori di programmazione durante la fase di scrittura, rigido nella sintassi
- **INDIPENDENZA DALLA PIATTAFORMA**: grazie alla JVM ogni applicazione, una volta compilata, potrà essere eseguita su qualsiasi piattaforma. Non necessita dunque di una modifica del codice.
- **JVM**: Java Virtual Machine è un software che svolge il **ruolo di interprete**.
  Java può in effetti essere un **linguaggio sia compilato che interpretato**. Dunque un codice sorgente(file.java) viene, una volta scritto, viene compilato in java e si crea un file in bytecode(suffisso.class). Dopodiché passa la parola alla JVM che interpreta il bytecode e lo esegue indipendentemente dal sistema operativo.



- **ORIENTATO AD OGGETTI:** Paradigmi fondamentali (ereditarietà, incapsulamento e polimorfismo)
- **FACILITÀ DI SVILUPPO:** facilita lo sviluppo dei software odierni
- **LIBRERIA E STANDARDIZZAZIONE:** possiede un'enorme libreria di classi standard . Possiamo utilizzare migliaia di soluzioni software già pronte per essere usate.
- **APERTO:** aperto a tanti ambienti di sviluppo, interagisce con gli altri linguaggi e tecnologie ed è open source.

AMBIENTI DI SVILUPPO - IDE

- Esistono vari ambienti di sviluppo, quello ufficiale è il **JDK** (java development kit) che offre **COMPILATORE JAVAC + JRE +JVM**

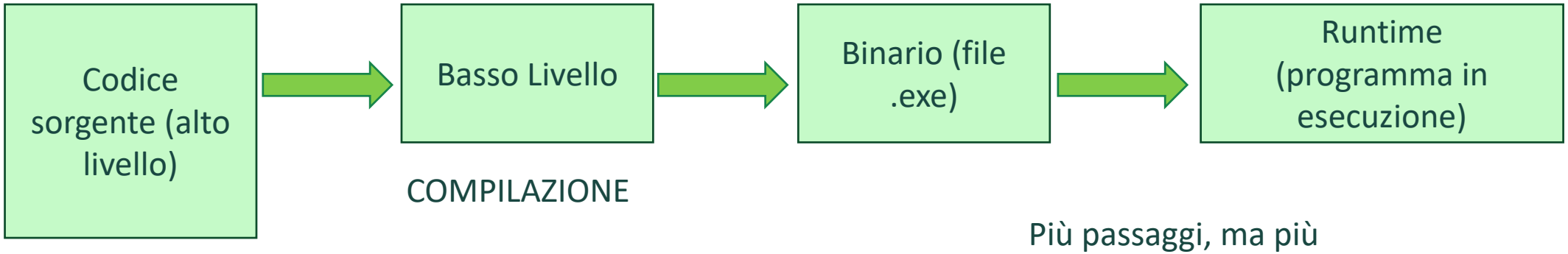
Da esso si puo compilare manualmente da cmd.

Ambienti di sviluppo con GUI invece:

- INTELLIJ
- ECLIPSE
- **NETBEANS**
- VS CODE

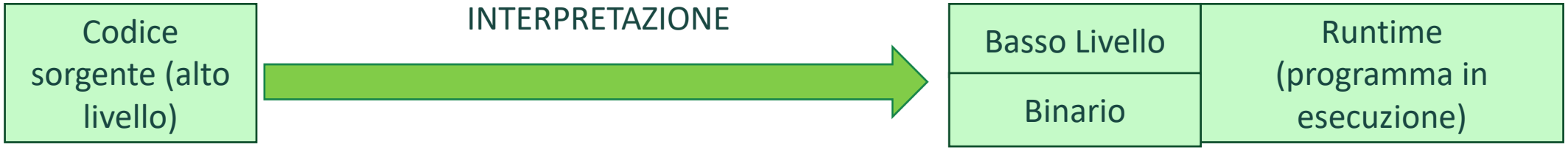
TIPI DI LINGUAGGIO

COMPILATO



Più passaggi, ma più leggera per il calcolatore

INTERPRETATO



TIPI DI LINGUAGGIO

COMPILATO

I linguaggi compilati vengono convertiti direttamente nel codice macchina che viene eseguito dal processore. Di conseguenza, tendono ad essere **più veloci ed efficienti da eseguire** rispetto ai linguaggi interpretati. Inoltre, permettono allo sviluppatore di avere un maggior controllo sugli aspetti legati all'hardware, come la gestione della memoria e l'uso della CPU.

Alcuni esempi di linguaggi compilati sono C, C++, Rust.

INTERPRETATO

Gli interpreti funzionano attraverso un programma che esegue ogni comando riga per riga al momento. I linguaggi interpretati erano notevolmente **più lenti dei linguaggi compilati**. Adesso, con lo sviluppo della compilazione just-in-time, il **divario sta diminuendo**.

Esempi di linguaggi interpretati sono PHP, Ruby, Python, e JavaScript.

TIPI DI LINGUAGGIO

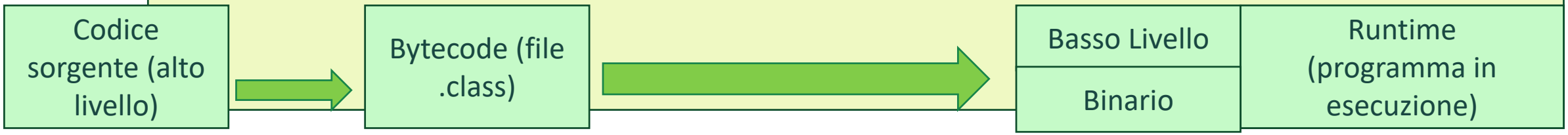
Linguaggi ibridi

linguaggio che viene prima **compilato**, dando origine ad un P-code, o **pseudo-codice,** il quale verrà a sua volta **interpretato**. La seconda fase di elaborazione svolge la funzione di macchina virtuale, o CPU generica. Sono linguaggi pseudocompilati **Perl, Java**.

In breve, in questi linguaggi il codice sorgente viene compilato in un codice intermedio simile all'ASM e dopo viene eseguito da un interprete in binario.

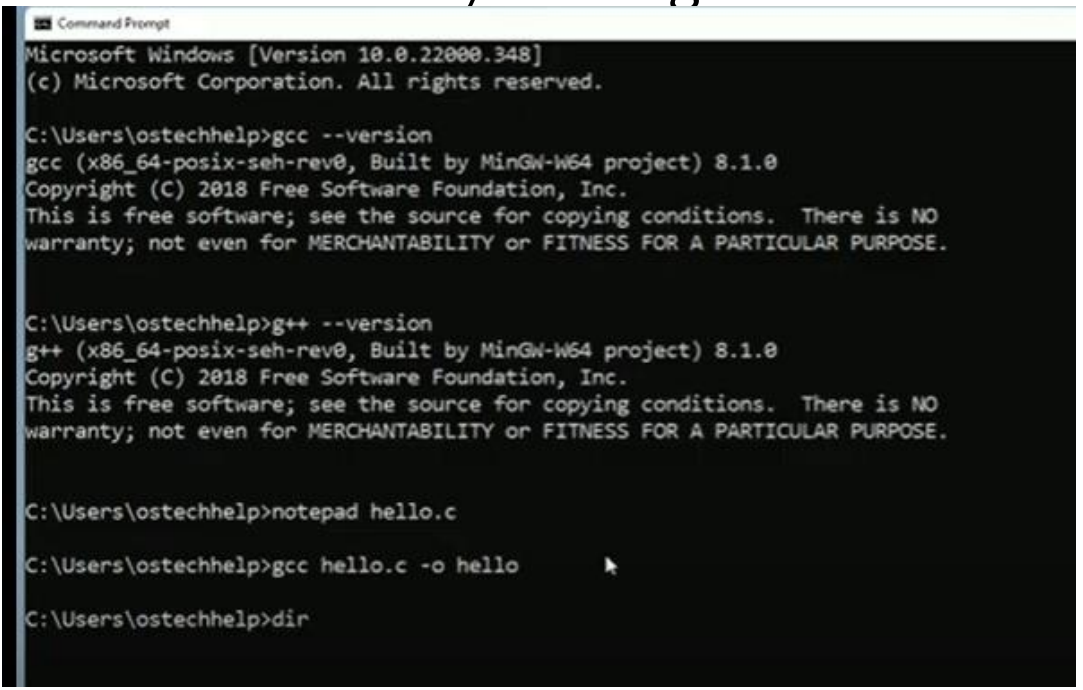
In questo modo in termini di velocità esecuzione questi linguaggi possiamo inserirli a metà tra i compilati e gli interpretati, con il bello di avere una grandissima **portabilità**.

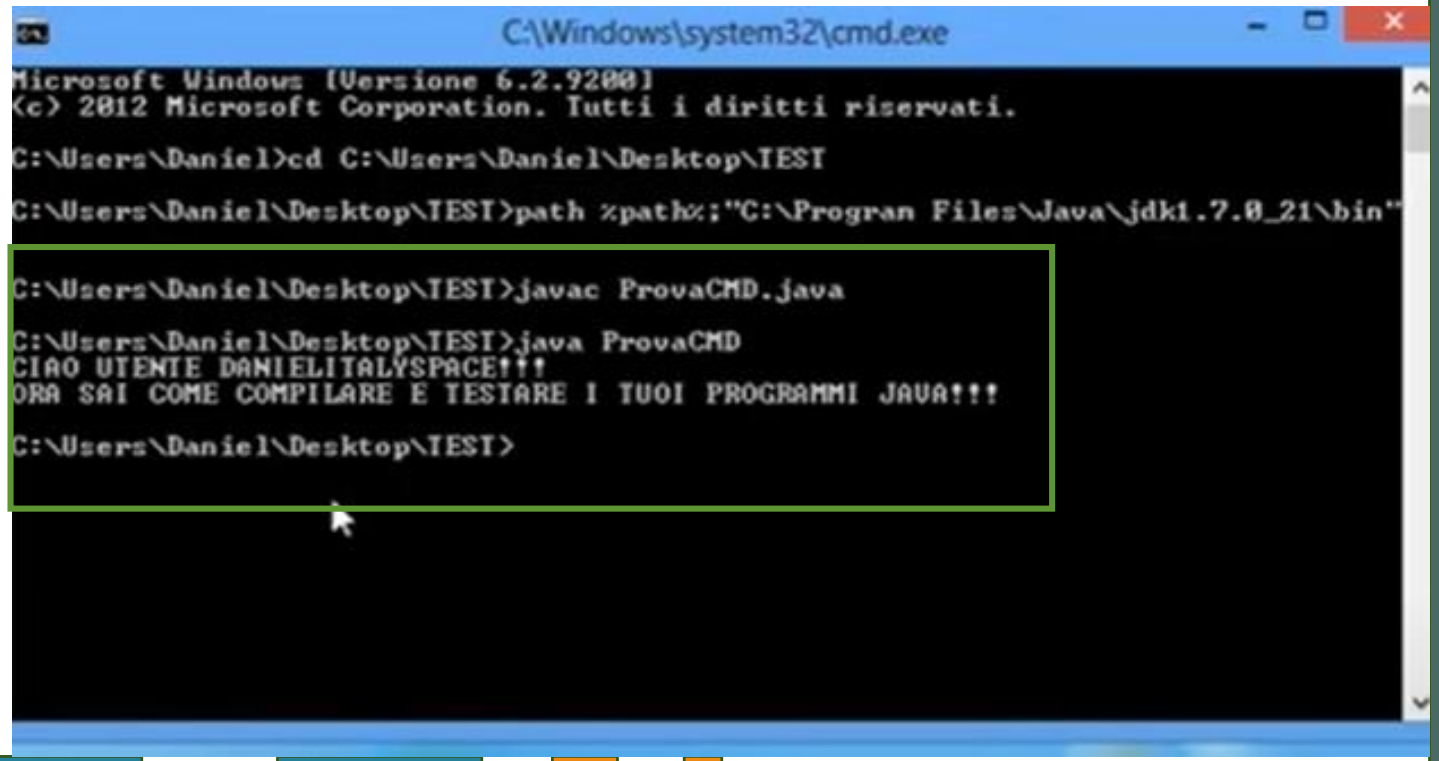
Un esempio può essere il Java dove il sorgente traduce il programma in linguaggio Java (.java) in un file bytecode (.class) e poi la Java Virtual Machine lo interpreta.



COMPILAZIONE TRAMITE CMD

- Ricordo che si può programmare anche usando un semplice editor di testo, salvando il file nella giusta estensione.
- Dopodichè si fa uso diretto del cmd o prompt dei comandi, dal quale selezionando la cartella, il file e il compilatore (gcc, g++, **javac**) e dando gli appositi comandi si può compilare il file (sia bytecode che codice macchina) ed eseguire.





LIBRERIE

- Java è un linguaggio che nasce ad oggetti (che tratteremo a oggetti)
- Gli oggetti nascono dalle cosiddette classi(esempio classe gatto, oggetto gatto_fufi)
- Le classi hanno attributi (varibili, stringhe, altri oggetti)
  e hanno metodi (le conosciamo come funzioni) associati alla classe stessa
- Quindi le librerie in java sono raccolte di metodi(funzioni di una certa classe) già preconfezionate e scritte in maniera procedurale che, alla loro chiamata, ci aiutano a scrivere i vari programmi.
- In particolare in Java per richiamare una libreria si usa la parola chiave **import**
- **Librerie principali:**

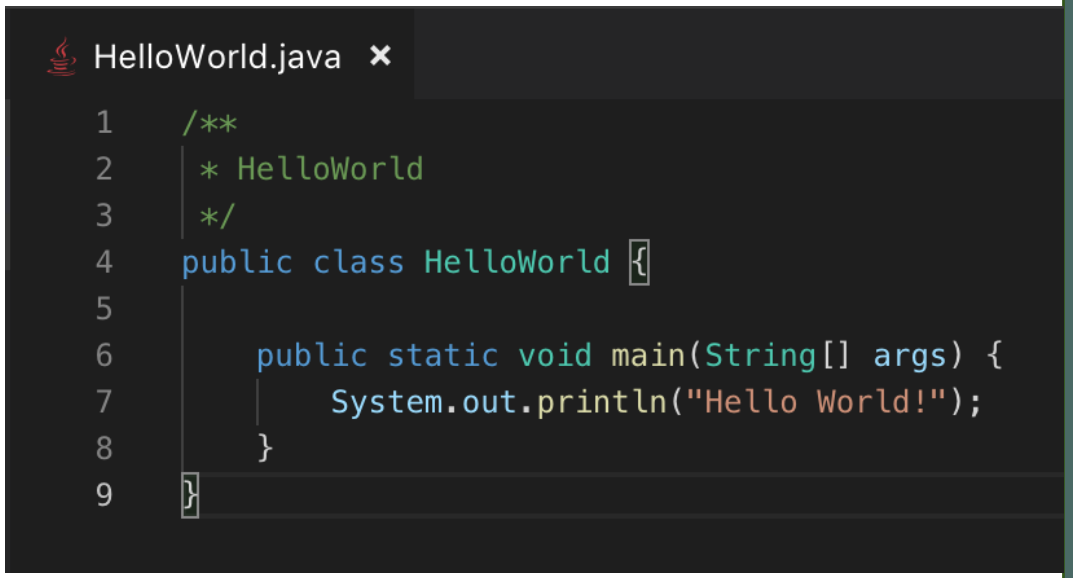
import java.util.Scanner **import java.lang.*** **import java.io.***

Da notare: **la Dot Notation e gli asterischi (all)**

PRIMO PROGRAMMA

- Il primo programma «Hello world» (un classico)

- Dalla **riga 1 alle 3**: un **commento /* … */**, non verrà letto dal sistema, verrà ignorato.



- **Riga 4**: stiamo definendo una **classe** (class) chiamata HelloWolrd. La classe è una **struttura di programmazione** tipica del linguaggio java. Tutte le istruzioni sono contenute in classi in questo linguaggio. Successivamente approfondiremo meglio il concetto di classe. Il nome della classe è modificabile.

Public invece è un modificatore d'accesso: la classe pubblica può essere accessibile da altri file.

Static invece vuole dire rendere UNIVCO un metodo (funzione) per quella classe e può essere **chiamato** senza creare altri oggetti.

Parametro: **String[] args,** serve per rivere input dalla riga di comando o cmd

Void si tratta del tipo di ritorno del metodo (che più avanti spiegheremo) ovvero il tipo di dato da restituire come output. Void in inglese vuol dire vuoto quindi nessun valore. Il metodo **main** non deve mai avere un tipo di ritorno diverso da **void.**

System.out.println è l'istruzione (**statement**) per stampare a video un messaggio su una riga.

Il tempo in cui il programma è in esecuzione viene detto **runtime**

HelloWorld.java

```java
/**
 * HelloWorld
 */
public class HelloWorld {

    public static void main(String[] args) {
        System.out.println("Hello World!");
    }
}
```

ESEMPIO PROGRAMMA INTERATTIVO

- **Import java.util.Scanner;**
  Questa istruzione permette di caricare all'interno del programma una classe scanner già definita all'interno delle librerie java. Per caricarla abbiamo usato la parola import. Tutte le classi della libreria java sono organizzate all'interno di cartelle chiamate **package(insieme di piu classi).**
- La cartella java.util contiene le classi di utilità come la classe Scanner che permette di acquisire in input da tastiera.
- **System.out**
  Java è un linguaggio a oggetti, perché le azioni vengono eseguite da elementi detti oggetti. Gli oggetti contengono metodi perché vengono creati partendo da classi.
  L'oggetto System.out è stato creato a partire da una classe. Questa definisce il metodo println che, quando invocato, esegue le istruzioni definite dalla classe. Per invocare un metodo si utlizza il punto «**.**» detto **dot notation.** Es: nomeOggetto.nomeMetodo (parametri);
  Ricorda che println stampa su una riga rispetto al solo print

```java

import java.util.Scanner;
class MoltiplicaDueInteri {

    Run | Debug
    public static void main (String args[]) {

        System.out.println(x:"Ciao so moltiplicare due numeri interi");
        System.out.println(x:"Scrivi il primo valore e premi invio");
        Scanner keyboardScanner = new Scanner(System.in);
        int input1;
        input1=keyboardScanner.nextInt();
        System.out.println(x:"Scrivi il secondo valore e premi invio");
        int input2 = keyboardScanner.nextInt();
        int risultato = input1*input2;
        System.out.println(x:"Il risultato è:");
        System.out.println(risultato);

    }
}
```

import viene usato per richiamare package(insieme di classi, oggetti e metodi) che sono esterni al package in cui si sta lavorando, Altrimenti usiamo **package** se interno

ESEMPIO INPUT

- **Scanner s1 = new Scanner (System.in)**
  In questa riga stiamo creando un **oggetto (in heap)** di nome *s1* che viene creato con Costruttore attraverso la parolina **new** e le parentesi **( )**. Sarà spiegato meglio nei prossimi capitoli.
- **int input1;**
  Qui dichiariamo una **variabile** di **tipo** intero, cioè un elemento di programmazione che permette di memorizzare dati. Int sta per **intero**(numeri interi). Mentre il dato contenuto all'interno della variabile prende il nome di **valore**. In questo caso non abbiamo assegnato valore. Lo avremmo potuto esprimere con il simbolo uguale es. int input1 = 8;
- **input1 = s1.nextInt()**
  prende dati in input e li salva nella variabile intera input1

Libreria: import java.util.Scanner

Metodi della classe Scanner per leggere dati dal terminale:

1. Lettura di stringhe e parole

- **next()**:
  - Legge un singolo token (parola), ignorando gli spazi successivi.
  - Si ferma al primo spazio o carattere non valido.

```
String parola = scanner.next();
```

- **nextLine()**:
  - Legge l'intera riga di input, compresi gli spazi.
  - Si ferma solo quando viene premuto "Invio".

```
String riga = scanner.nextLine();
```

2. Lettura di numeri interi e decimali

- **nextInt()**:
  - Legge il prossimo intero.

```
int numero = scanner.nextInt();
```

- **nextLong()**:
  - Legge il prossimo valore di tipo long.

```
long grandeNumero = scanner.nextLong();
```

- **nextFloat()**:
  - Legge il prossimo valore decimale di tipo float.

```
float numeroVirgola = scanner.nextFloat();
```

- **nextDouble()**:
  - Legge il prossimo valore decimale di tipo double.

```
double numeroGrandeVirgola = scanner.nextDouble();
```

3. Lettura di valori booleani

- **nextBoolean()**:
  - Legge un valore booleano (true o false).

```
boolean valore = scanner.nextBoolean();
```

ESEMPIO PROGRAMMA INTERATTIVO

- **Import java.util.Scanner;**
  Questa istruzione permette di caricare all'interno del programma una classe scanner già definita all'interno delle librerie java. Per caricarla abbiamo usato la parola import. Tutte le classi della libreria java sono organizzate all'interno di cartelle chiamate **package.** La cartella java.util contiene le classi di utilità come la classe Scanner che permette di acquisire in input da tastiera.
- **System.out**
  Java è un linguaggio a oggetti, perché le azioni vengono eseguite da elementi detti oggetti. Gli oggetti contengono metodi perché vengono creati partendo da classi. L'oggetto System.out è stato creato a partire da una classe. Questa definisce il metodo println che, quando invocato, esegue le istruzioni definite dalla classe. Per invocare un metodo si utlizza il punto «**.**» detto **dot notation.** Es: nomeOggetto.nomeMetodo (parametri);
  Ricorda che println stampa su una riga rispetto al solo print

```java
import java.util.Scanner;
class MoltiplicaDueInteri {

    Run | Debug
    public static void main (String args[]) {

        System.out.println(x:"Ciao so moltiplicare due numeri interi");
        System.out.println(x:"Scrivi il primo valore e premi invio");
        Scanner keyboardScanner = new Scanner(System.in);
        int input1;
        input1=keyboardScanner.nextInt();
        System.out.println(x:"Scrivi il secondo valore e premi invio");
        int input2 = keyboardScanner.nextInt();
        int risultato = input1*input2;
        System.out.println(x:"Il risultato è:");
        System.out.println(risultato);

    }
}
```

import viene usato per richiamare package(insieme di classi, oggeti e metodi) che sono esterni al package in cui si sta lavorando, Altrimenti usiamo **package** se interno

OUTPUT

- **Stampa a schermo e vai a capo**

```
System.out.println("This sentence will work!");
System.out.println("This sentence will work!");
```

- **Stampa a schermo senza andare a capo, stessa riga**

```
System.out.print("This sentence will work!");
System.out.print("This sentence will work!");
```

Operatori logici di confronto

SIMBOLO	SIGNIFICATO	SIMBOLO	SIGNIFICATO
=	ASSEGNAZIONE	&	AND BINARIO
==	UGUALE, IDENTICO	>	MAGGIORE
!=	DIVERSO	<	MINORE
||	OR LOGICO	>=	MAGGIORE O UGUALE
&&	AND LOGICO	<=	MINORE O UGUALE
|	OR BINARIO		

Caratteri speciali

CARATTERE SPECIALE	DESCRIZIONE
\n	Riga a capo
\t	Tabulazione
\f	Salto pagina
\”	Doppi apici
\\	Barra contraria
\a	Segnalazione acustica
\’	Apice singolo
\?	Punto interrogativo

Tipi di variabile/identificatori

Data Type	Size	Description
`byte`	1 byte	Stores whole numbers from -128 to 127
`short`	2 bytes	Stores whole numbers from -32,768 to 32,767
`int`	4 bytes	Stores whole numbers from -2,147,483,648 to 2,147,483,647
`long`	8 bytes	Stores whole numbers from -9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
`float`	4 bytes	Stores fractional numbers. Sufficient for storing 6 to 7 decimal digits
`double`	8 bytes	Stores fractional numbers. Sufficient for storing 15 decimal digits
`boolean`	1 bit	Stores true or false values
`char`	2 bytes	Stores a single character/letter or ASCII values

Per inserire una **COSTANTE** basta usare la parola final di fronte al nome di variabile es: **final double PIGREC=3,1415**

Altri operatori

Operator	Name	Description	Example
+	Addition	Adds together two values	x + y
-	Subtraction	Subtracts one value from another	x - y
*	Multiplication	Multiplies two values	x * y
/	Division	Divides one value by another	x / y
%	Modulus	Returns the division remainder	x % y
++	Increment	Increases the value of a variable by 1	++x
--	Decrement	Decreases the value of a variable by 1	--x

Valgono anche : += -= /= *=

Pre Incremento e post incremento

- **Incremento classico**

  i=3;

- **i++ ; -> i sarà 4**

```
int x = 5;
System.out.println(++x);
```

Output:

```
6
```

- **Durante l'assegnazione di variabili si ha che:**
  - **Preincremento**

    x=3;
    y= (++x)+1;

    Risultato in memoria
    y=5 e x=4

  - Postincremento

    x=3;
    y= (x++)+1;

    Risultato in memoria

    x=4 e y=4

```
int x = 5;
System.out.println(x++);
System.out.println(x);
```

Output:

```
5
6
```

Casting in Java

- **Conversione di tipo di dato**

```
float a=3,5;
int b;
b= (int) a;
```

```
int x=3;
int b= (float) x;
```

Classe math

- **Richiamare la classe math per le operazioni matematiche o per la generazione di numeri casuali**

```
import java.math.*
```

```
Math.abs(a);

Math.max(a, b, c);

Math.min(a, b);

Math.sqrt(a);
```

```
Math.random();

int randomNum = (int)(Math.random() * 101);  // 0 to 100
   System.out.println(randomNum);
```

Numeri causali

* (max - min + 1)) + min

In Java puoi usare Math.random().

La formula generale è:

int n = (int)(Math.random() * (max - min + 1)) + min;

Da 0 a 10

int n = (int)(Math.random() * 11);

Da 1 a 50

int n = (int)(Math.random() * 50) + 1;

Da 55 a 200

int n = (int)(Math.random() * 146) + 55;

La regola da ricordare è:

(min, max) → (int)(Math.random() * (max - min + 1)) + min

Quindi da 1 a 10

int a= (int) (Math.random() * 10)+1;

invece da 55 a 200

int b=(int) (Math.random()*146)+55;

If else

- **Richiamare la classe**

```
int x = 20;
int y = 18;
if (x > y) {
  System.out.println("x is greater than y");
}


else if (x<y){
 System.out.println(“y is greater than y");


}
Else  System.out.println(“they are euguals");
```

If else operatore ternario

- **Operatore ternario, scrittura ridotta dell'ifelse**

```
variable = (condition) ? expressionTrue :  expressionFalse;
```

```
int time = 20;
String result = (time < 18) ? "Good day." : "Good evening.";
System.out.println(result);
```

If annidato

```java
int eta = 20;

if (eta >= 18) {
    if (eta >= 65) {
        System.out.println("Anziano");
    } else {
        System.out.println("Adulto");
    }
}
```

Con Operatori logici

```java
int eta = 20;

if (eta >= 18 && eta < 65) {
    System.out.println("Adulto");
}
```

Switch

- **Selezione multipla**

```
String giorno = "Lunedì";

switch (giorno) {
    case "Lunedì":
        System.out.println("Inizio settimana");
        break;

    case "Venerdì":
        System.out.println("Fine settimana");
        break;

    case "Sabato":
    case "Domenica":
        System.out.println("Weekend");
        break;

    default:
        System.out.println("Giorno normale");
}
```

While e Do..While

```java
int i = 0;

while (i < 5) {
    System.out.println(i);
    i++;
}
```

Prima **controlla la condizione**, poi esegue il blocco.

```java
int i = 0;

do {
    System.out.println(i);
    i++;
} while (i < 5);
```

Prima **esegue il blocco**, poi controlla la condizione.

For e Foreach

for → uso l'indice for each → uso direttamente l'elemento

```java
int[] numeri = {10, 20, 30, 40};

for (int i = 0; i < numeri.length; i++) {
    System.out.println(numeri[i]);
}
```

```java
int[] numeri = {10, 20, 30, 40};

for (int numero : numeri) {
    System.out.println(numero);
}
```

for-each serve per scorrere tutti gli elementi
del vettore, dall'inizio alla fine, senza gestire l'indice.
incrementando soltanto di 1

Chiamate di funzione

Chiamate di metodi in classi diverse

In una classe diversa: devo creare un oggetto

Java

```java
class A {
    void metodo() {
        System.out.println("Ciao");
    }
}
```

Java

```java
class B {
    public static void main(String[] args) {
        A oggetto = new A();
        oggetto.metodo();
    }
}
```

Chiamate di metodi della stessa classe

Nella stessa classe: posso chiamarlo direttamente, senza oggetto.

Java

```java
class A {
    void metodo1() {
        metodo2();
    }

    void metodo2() {
        System.out.println("Ciao");
    }
}
```

Array

Vettori monodimensionali

```java
String[] cars = {"Volvo", "BMW", "Ford", "Mazda"};
cars[0] = "Opel";
System.out.println(cars[0]);
```

```java
String[] cars = {"Volvo", "BMW", "Ford", "Mazda"};

for (int i = 0; i < cars.length; i++) {
  System.out.println(cars[i]);
}
```

new -> serve per creare una nuova struttura dati in memoria (oggetti, vettori).

Matrici

```java
int[][] myNumbers = { {1, 4, 2}, {3, 6, 8} };

System.out.println(myNumbers[0][1]); // Outputs 4
```



Eccezioni

Le eccezioni **sono eventi anomali** che si verificano durante l'esecuzione del programma e try-catch-finally servono a gestirli senza interrompere il programma in modo **imprevisto.**

```
try {
    // codice che potrebbe generare un'eccezione
}
catch (Exception e) {
    // gestione dell'errore
}
finally {
    // viene eseguito comunque
}
```

```
int a = 10;
int b = 0;

try {
    System.out.println(a / b);
}
catch (Exception e) {
    System.out.println("Errore!");
}
finally {
    System.out.println("Fine del programma");
}
```

A cosa servono:

try → contiene il codice che potrebbe generare un'eccezione.

catch → intercetta e gestisce l'eccezione.

finally → viene eseguito comunque, sia che si verifichi un'eccezione sia che non si verifichi.
Finally è facoltativo

Altro metodo di Input con streamreader

Anche se **InputStreamReader** converte i byte in caratteri,
legge un carattere alla volta.

Nella maggior parte dei programmi, però, non interessa
leggere una lettera per volta: si vuole leggere una riga intera.
Per questo motivo si aggiunge un BufferedReader,
che raccoglie i caratteri e permette di usare il metodo readLine().

Devi leggere un grosso flusso di testo velocemente?→
BufferedReader è più conveniente.Devi costruire un componente
che riceve dati da una sorgente di byte?→ InputStreamReader è il
componente giusto.

```java
import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.io.IOException;

public class Main {

    public static void main(String[] args) throws IOException {

        InputStreamReader isr = new InputStreamReader(System.in);
        BufferedReader br = new BufferedReader(isr);

        System.out.print("Inserisci il tuo nome: ");

        String nome = br.readLine();

        System.out.println("Ciao " + nome);
    }
}
```